



International Islamic University Chittagong

Department of Computer Science and Engineering

Course: Operating Systems Lab (CSE-3632)

Session: Spring 2026 Semester: 6th Section: 6BF

StealthGuard

Intrusion Detection System with File Integrity Monitoring
Lab Project Report

Team Members:

Tahasina Tasnim Afra	C233456
Nusrat Jahan	C233464
Asliraf Samaylan	C233446

Supervisor:

Ms. Sauda Adiv Hanum
Lecturer, Department of CSE, IIUC

Date of Submission: August 1, 2026

Contents

1	Introduction	2
2	Objectives	2
3	Application	2
4	Features	3
4.1	Automated Screenshot Capture	3
4.2	Timestamp-Based File Naming	4
4.3	Process Snapshot Monitoring	5
4.4	Zero-Byte Corruption Detection	6
4.5	SHA-256 Cryptographic Hashing	7
4.6	Hash-Based Change Detection	7
4.7	Persistent Audit Logging	8
4.8	Manual Trigger via CLI Flag	9
4.9	Automatic Loop Mode	10
4.10	Zip and Email Report	11
4.11	Screen Change Alert Email	12
4.12	Real-Time Folder Watch and File Modification Alert	13
5	Conclusion	15

1. Introduction

In modern computer security architectures, real-time operating system monitoring and post-event auditing are critical defense pillars. Operating systems continuously experience hidden state changes, file access modifications, and background application executions. For a specialized security platform like Kali Linux, automating the capture and evaluation of these internal events is both useful and essential to maintain system integrity.

StealthGuard is a lightweight automated security application designed to bridge the gap between low-level core system operations and high-level analytical data verification. The system is structured into two native execution layers:

- **Shell Automation Engine** (`stealthguard.sh`): Works directly with the operating system to capture system snapshots, monitor processes, and trigger actions automatically or on-demand via command-line flags.
- **Python Integrity Auditor** (`verify.py`): Interfaces with file storage metrics to verify file integrity, detect corruption, calculate cryptographic hashes, and maintain a permanent timestamped audit log.

Together, these two layers implement a clean **Trigger** → **Capture** → **Verify** → **Record** pipeline that aligns directly with core Operating System concepts including process management, file systems, inter-process communication, and persistent logging.

2. Objectives

The main objectives of this project are:

- To develop an automated OS monitoring system using Bash and Python on Kali Linux.
- To capture screenshots and process information automatically and manually.
- To verify file integrity using SHA-256 hashing and detect screen changes.
- To detect corrupted or empty files using zero-byte file checking.
- To maintain timestamped audit logs of all system activities.
- To monitor and record running processes during each capture.
- To send email alerts when screen changes or file modifications are detected.
- To compress and email captured screenshots, process snapshots, and audit logs.
- To monitor the `/SecurityVault` folder in real time using `inotifywait` and detect unauthorized file changes.
- To demonstrate communication between Shell scripting and Python.

3. Application

StealthGuard has several practical real-world applications in the field of system security and digital forensics, as described below:

1. Unauthorized Access Detection

If an unauthorized person gains access to a system, StealthGuard silently captures screenshot evidence of what was on screen, records all running processes at that exact moment, and logs a timestamped audit entry—all without the intruder's knowledge.

2. Digital Forensics and Evidence Collection

The persistent audit log and screenshot archive provide timestamped forensic evidence that can be used to reconstruct a timeline of events during a security incident. The SHA-256 hash stored for each capture ensures the integrity of evidence files.

3. File Tampering Detection

Using real-time folder monitoring via `inotifywait`, StealthGuard detects any modification, creation, or deletion of files inside the `/SecurityVault` directory and immediately sends an email alert, making it suitable for protecting sensitive log files and captured evidence.

4. Remote Monitoring and Reporting

The zip-and-email feature allows a system administrator to receive a complete archive of all captured screenshots, process snapshots, and audit logs directly to their email inbox, enabling remote monitoring without physical access to the machine.

5. Operating System Education

This project serves as a practical demonstration of core Operating System concepts including process management, file system operations, inter-process communication, cryptographic integrity verification, and shell scripting.

4. Features

StealthGuard implements a total of 13 features divided between the Shell and Python layers. Each feature is described below with its code and expected output.

4.1. Automated Screenshot Capture

Description: When triggered, StealthGuard uses the `scrot` tool to silently capture a full screenshot of the Kali Linux desktop and save it to the `/SecurityVault/` directory. This forms the core evidence capture mechanism of the system.

Code (stealthguard.sh):

```
1 scrot "$FILENAME"
2
3 if [ $? -eq 0 ]; then
4     echo -e "${GREEN}[OK] Screenshot saved: capture_${TIMESTAMP}.png${NC}"
5 else
6     echo -e "${RED}[ERROR] Screenshot failed.${NC}"
7     echo "[$(date)] CAPTURE FAILED" >> "$LOG"
8     return 1
9 fi
```

Listing 1: Screenshot Capture

Output:

```
(kali㉿kali)-[~/stealthguard]
$ ls /SecurityVault/

(kali㉿kali)-[~/stealthguard]
$ ./stealthguard.sh --capture

=====
StealthGuard - IDS System
IIUC CSE | OS Lab Project
=====

[OK] /SecurityVault exists.
[MODE] Manual capture.

[17:02:41] Starting capture ...
[OK] Screenshot saved: capture_2026-07-31_17-02-41.png
[OK] Process snapshot saved.
[INFO] Calling Python verifier ...
[OK] File size: 650384 bytes
[INFO] SHA-256: 9dd34aba61b5cf799d6c ...
[INFO] FIRST RUN - No previous hash.
[OK] Audit log updated.

(kali㉿kali)-[~/stealthguard]
$
```

Figure 1: Screenshot capture output in terminal

4.2. Timestamp-Based File Naming

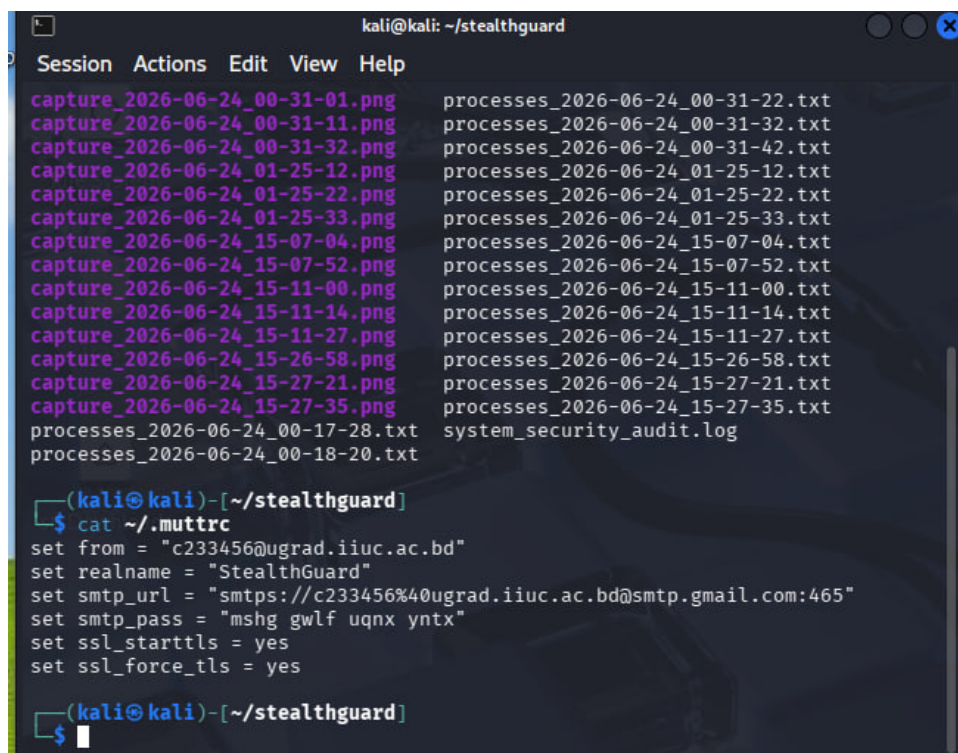
Description: Every captured file receives a unique filename generated from the current system date and time. This prevents any two files from overwriting each other and creates a natural chronological record of all captures.

Code (stealthguard.sh):

```
1 TIMESTAMP=$(date +"%Y-%m-%d_%H-%M-%S")
2 FILENAME="$VAULT/capture_${TIMESTAMP}.png"
3 PROC_FILE="$VAULT/processes_${TIMESTAMP}.txt"
```

Listing 2: Timestamp Filename Generation

Output:



```
kali@kali: ~/stealthguard
Session Actions Edit View Help
capture_2026-06-24_00-31-01.png processes_2026-06-24_00-31-22.txt
capture_2026-06-24_00-31-11.png processes_2026-06-24_00-31-32.txt
capture_2026-06-24_00-31-32.png processes_2026-06-24_00-31-42.txt
capture_2026-06-24_01-25-12.png processes_2026-06-24_01-25-12.txt
capture_2026-06-24_01-25-22.png processes_2026-06-24_01-25-22.txt
capture_2026-06-24_01-25-33.png processes_2026-06-24_01-25-33.txt
capture_2026-06-24_15-07-04.png processes_2026-06-24_15-07-04.txt
capture_2026-06-24_15-07-52.png processes_2026-06-24_15-07-52.txt
capture_2026-06-24_15-11-00.png processes_2026-06-24_15-11-00.txt
capture_2026-06-24_15-11-14.png processes_2026-06-24_15-11-14.txt
capture_2026-06-24_15-11-27.png processes_2026-06-24_15-11-27.txt
capture_2026-06-24_15-26-58.png processes_2026-06-24_15-26-58.txt
capture_2026-06-24_15-27-21.png processes_2026-06-24_15-27-21.txt
capture_2026-06-24_15-27-35.png processes_2026-06-24_15-27-35.txt
processes_2026-06-24_00-17-28.txt system_security_audit.log
processes_2026-06-24_00-18-20.txt

(kali@kali)~[~/stealthguard]
$ cat ~/.muttrc
set from = "c233456@ugrad.iuc.ac.bd"
set realname = "StealthGuard"
set smtp_url = "smtps://c233456%40ugrad.iuc.ac.bd@smtp.gmail.com:465"
set smtp_pass = "mshg gwlf uqnx yntx"
set ssl_starttls = yes
set ssl_force_tls = yes

(kali@kali)~[~/stealthguard]
$
```

Figure 2: Files saved with timestamp names in /SecurityVault

4.3. Process Snapshot Monitoring

Description: Alongside every screenshot, StealthGuard captures a snapshot of the top 10 running processes sorted by CPU usage. This provides context for detecting suspicious programs that may be running in the background at the time of an intrusion.

Code (stealthguard.sh):

```
1 PROC_FILE="$VAULT/processes_$TIMESTAMP.txt"
2 echo "=== Top 10 Processes at $TIMESTAMP ===" > "$PROC_FILE"
3 ps aux --sort=-%cpu | head -11 >> "$PROC_FILE"
4 echo -e "${GREEN}[OK] Process snapshot saved.${NC}"
```

Listing 3: Process Snapshot

Output:

```

(kali@kali)-[~]
$ cd /SecurityVault/

(kali@kali)-[/SecurityVault]
$ cat /SecurityVault/processes_*.txt
=== Top 10 Processes at 2026-07-31_17-02-41 ===
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
kali      4472  200  0.2   9504   4228 pts/0    R+   17:02   0:00 ps aux --s
ort=-%cpu
root       768   3.2  6.5 683660 131388 tty7      Ssl+ 16:55   0:14 /usr/lib/x
org/Xorg :0 -seat seat0 -auth /var/run/lightdm/root/:0 -nolisten tcp vt7 -nov
tswitch
kali      1284   1.2  2.2 505932 45048 ?          Sl   16:57   0:04 Thunar --d
aemon
kali      1929   1.2  0.3  10340   6408 pts/0    Ss   16:57   0:03 /usr/bin/z
sh
kali      1301   1.1  2.5 296936 51592 ?          Sl   16:57   0:03 /usr/lib/x
86_64-linux-gnu/xfce4/panel/wrapper-2.0 /usr/lib/x86_64-linux-gnu/xfce4/panel
/plugins/libcpugraph.so 13 25165836 cpugraph CPU Graph Graphical representati
on of the CPU load

```

Figure 3: Process snapshot file contents

4.4. Zero-Byte Corruption Detection

Before processing any captured screenshot, the Python layer checks the file size using `os.path.getsize()`. If the file size equals zero, it means the capture was empty and corrupted. StealthGuard immediately flags it as a failed capture and writes a failure entry to the audit log instead of recording corrupt data as valid evidence. In a security monitoring system, recording a corrupt empty file as valid evidence is dangerous. It could mean a real intrusion event was missed because the capture failed silently. By detecting zero-byte files immediately, StealthGuard ensures only verified, valid captures are accepted into the audit record

Code (verify.py):

```

1 size = os.path.getsize(screenshot)
2 if size == 0:
3     msg = f"FAILURE - Zero-byte file (corrupt): {filename}"
4     print(f"{RED}[FAIL] {msg}{NC}")
5     log(msg)
6     return False
7
8 print(f"{GREEN}[OK] File size: {size} bytes{NC}")

```

Listing 4: Zero-Byte Detection

Output:

```

(kali@kali)-[~/stealthguard]
$ touch /SecurityVault/fake_capture.png

(kali@kali)-[~/stealthguard]
$ python3 ~/stealthguard/verify.py /SecurityVault/fake_capture.png /Securit
yVault/processes_*.txt
[FAIL] FAILURE - Zero-byte file (corrupt): fake_capture.png

```

Figure 4: Zero-byte detection test output

4.5. SHA-256 Cryptographic Hashing

Description: Each screenshot is passed through the SHA-256 hashing algorithm, producing a unique 64-character fingerprint. This hash uniquely identifies the exact visual content of the capture and is used for change detection in the next step.

Code (verify.py):

```
1 def get_sha256(filepath):
2     sha256 = hashlib.sha256()
3     with open(filepath, "rb") as f:
4         for chunk in iter(lambda: f.read(4096), b''):
5             sha256.update(chunk)
6     return sha256.hexdigest()
```

Listing 5: SHA-256 Hash Calculation

The SHA-256 hash is visibly displayed in the terminal output during each capture cycle, as shown in Figure 1 (Manual Capture Output). The hash appears as a unique 64-character string, for example:

```
1 [INFO] SHA-256: 66362731d9aaf9ab57a8...
```

Refer to **Figure 1** for the complete terminal output showing the SHA-256 hash generated during the capture process.

4.6. Hash-Based Change Detection

Description: Every screenshot captured by StealthGuard is passed through the SHA-256 hashing algorithm to produce a unique 64-character fingerprint of the image. This fingerprint is then compared against the hash stored from the previous capture cycle, which is saved in a hidden file called `.previous_hash` inside `/SecurityVault/`.

If the two hashes are identical, the screen has not changed. If they differ even by a single pixel — the system immediately flags a `ALERT - Screen change detected!` event. This mechanism allows StealthGuard to detect unauthorized activity even if the intruder closes everything before the next capture, as long as the screen visually changed at any point during the cycle.

There are three possible outcomes of each hash comparison:

- **FIRST RUN:** No `.previous_hash` file exists yet. This is the very first capture. No comparison is made and the current hash is saved for the next cycle.
- **NO CHANGE:** The current hash matches the previous hash exactly. The screen content is identical between captures. No alert is triggered.
- **ALERT:** The current hash is different from the previous hash. The screen changed between captures. An alert is logged and an email notification is sent automatically.

Code (verify.py):

```
1 if previous_hash is None:
2     status = "FIRST RUN - No previous hash."
3     print(f"{YELLOW}[INFO] {status}{NC}")
4 elif current_hash == previous_hash:
```

```

5     status = "NO CHANGE - Screen identical to last capture."
6     print(f"{GREEN}[OK] {status}{NC}")
7 else:
8     status = "ALERT - Screen change detected!"
9     print(f"{YELLOW}[ALERT] {status}{NC}")
10
11 save_hash(current_hash)

```

Listing 6: SHA-256 Hash Comparison and Change Detection

How the hash is stored and loaded:

```

1 def load_previous_hash():
2     if os.path.exists(PREV_HASH_FILE):
3         with open(PREV_HASH_FILE, "r") as f:
4             return f.read().strip()
5     return None
6
7 def save_hash(h):
8     with open(PREV_HASH_FILE, "w") as f:
9         f.write(h)

```

Listing 7: Saving and Loading Previous Hash

Example Output:

On the first capture, the system produces:

```

1 [INFO] SHA-256: 66362731d9aaf9ab57a8...
2 [INFO] FIRST RUN - No previous hash.
3 [OK] Audit log updated.

```

Listing 8: First Run Output

On the second capture after any screen change:

```

1 [INFO] SHA-256: c705d3b533c50bb91c45...
2 [ALERT] ALERT - Screen change detected!
3 [ALERT] Sending change detection email...
4 [OK] Alert email sent to recipient@gmail.com!

```

Listing 9: Change Detection Output

Notice the SHA-256 hash changed completely between the two captures — 66362731d9... became c705d3b533... — proving that even a small visual change on screen produces an entirely different hash fingerprint.

4.7. Persistent Audit Logging

Description: Every event — file name, size, SHA-256 hash, change status, and full process list — is written permanently to `system_security_audit.log` using append mode. This file grows over time and is never overwritten, forming a complete forensic history.

Code (verify.py):

```

1 def log(message):
2     timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
3     with open(LOG_FILE, "a") as f:

```

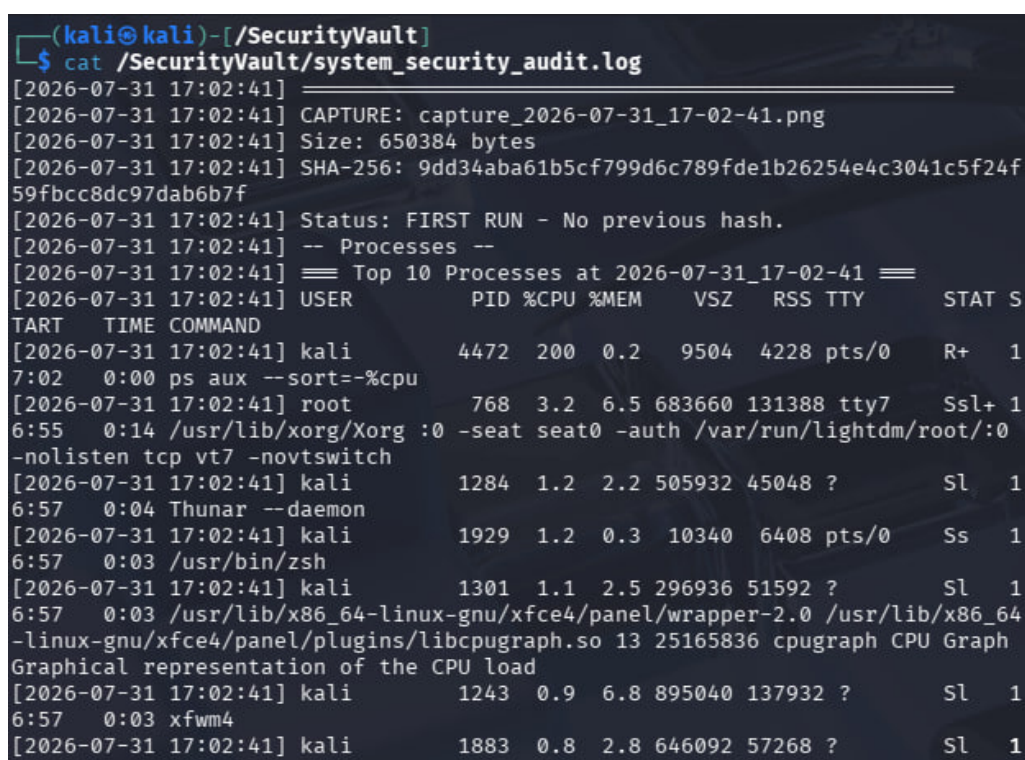
```

4         f.write(f"[{timestamp}] {message}\n")
5
6     log("=" * 50)
7     log(f"CAPTURE: {filename}")
8     log(f"Size: {size} bytes")
9     log(f"SHA-256: {current_hash}")
10    log(f"Status: {status}")
11    log("-- Processes --")
12    for line in processes.splitlines():
13        log(line)
14    log("=" * 50)

```

Listing 10: Audit Log Writing

Output:



```

(kali@kali)-[/SecurityVault]
$ cat /SecurityVault/system_security_audit.log
[2026-07-31 17:02:41] =====
[2026-07-31 17:02:41] CAPTURE: capture_2026-07-31_17-02-41.png
[2026-07-31 17:02:41] Size: 650384 bytes
[2026-07-31 17:02:41] SHA-256: 9dd34aba61b5cf799d6c789fde1b26254e4c3041c5f24f59fbcc8dc97dab6b7f
[2026-07-31 17:02:41] Status: FIRST RUN - No previous hash.
[2026-07-31 17:02:41] -- Processes --
[2026-07-31 17:02:41] == Top 10 Processes at 2026-07-31_17-02-41 ==
[2026-07-31 17:02:41] USER          PID %CPU %MEM    VSZ   RSS TTY      STAT S
TART   TIME COMMAND
[2026-07-31 17:02:41] kali            4472  200  0.2   9504   4228 pts/0    R+   1
7:02   0:00 ps aux --sort=-%cpu
[2026-07-31 17:02:41] root             768   3.2  6.5 683660 131388 tty7      Ssl+  1
6:55   0:14 /usr/lib/xorg/Xorg :0 -seat seat0 -auth /var/run/lightdm/root/:0
-nolisten tcp vt7 -novtswitch
[2026-07-31 17:02:41] kali            1284   1.2  2.2 505932 45048 ?        Sl    1
6:57   0:04 Thunar --daemon
[2026-07-31 17:02:41] kali            1929   1.2  0.3  10340   6408 pts/0    Ss    1
6:57   0:03 /usr/bin/zsh
[2026-07-31 17:02:41] kali            1301   1.1  2.5 296936 51592 ?        Sl    1
6:57   0:03 /usr/lib/x86_64-linux-gnu/xfce4/panel/wrapper-2.0 /usr/lib/x86_64
-linux-gnu/xfce4/panel/plugins/libcpugraph.so 13 25165836 cpugraph CPU Graph
Graphical representation of the CPU load
[2026-07-31 17:02:41] kali            1243   0.9  6.8 895040 137932 ?        Sl    1
6:57   0:03 xfwm4
[2026-07-31 17:02:41] kali            1883   0.8  2.8 646092 57268 ?        Sl    1

```

Figure 5: Audit log contents showing full event record

4.8. Manual Trigger via CLI Flag

Description: Running `./stealthguard.sh --capture` triggers a single manual capture cycle on demand. This is implemented using Bash argument handling where `$1` reads the first word typed after the command.

Code (stealthguard.sh):

```

1 if [ "$1" == "--capture" ]; then
2     echo -e "${YELLOW}[MODE] Manual capture.${NC}"
3     do_capture

```

Listing 11: Manual Trigger Argument

Output:

```
(kali㉿kali)-[~/stealthguard]
$ ls /SecurityVault/

(kali㉿kali)-[~/stealthguard]
$ ./stealthguard.sh --capture

=====
StealthGuard - IDS System
IIUC CSE | OS Lab Project
=====

[OK] /SecurityVault exists.
[MODE] Manual capture.

[17:02:41] Starting capture ...
[OK] Screenshot saved: capture_2026-07-31_17-02-41.png
[OK] Process snapshot saved.
[INFO] Calling Python verifier ...
[OK] File size: 650384 bytes
[INFO] SHA-256: 9dd34aba61b5cf799d6c ...
[INFO] FIRST RUN - No previous hash.
[OK] Audit log updated.

(kali㉿kali)-[~/stealthguard]
$
```

Figure 6: Manual capture triggered with `--capture` flag

4.9. Automatic Loop Mode

Description: Running `./stealthguard.sh` with no arguments enters auto mode. The system runs a complete capture cycle every 10 seconds indefinitely using a `while true` loop and `sleep` command. Press `Ctrl+C` to stop.

Code (stealthguard.sh):

```
1 else
2     echo -e "${YELLOW}[MODE] Auto mode - every ${INTERVAL}s.${NC}"
3     while true; do
4         do_capture
5         echo -e "${CYAN}[INFO] Waiting ${INTERVAL} seconds...${NC}"
6         sleep $INTERVAL
7     done
8 fi
```

Listing 12: Auto Loop Mode

Output:

```
(kali㉿kali)-[~/stealthguard]
$ ./stealthguard.sh

=====
StealthGuard - IDS System
IIUC CSE | OS Lab Project
=====

[OK] /SecurityVault exists.
[MODE] Auto mode - every 10 seconds. Ctrl+C to stop.
      Press Ctrl+C to stop.

[21:52:32] Starting capture ...
[OK] Screenshot saved: capture_2026-07-31_21-52-32.png
[OK] Process snapshot saved.
[INFO] Calling Python verifier
```

Figure 7: Auto mode running three consecutive capture cycles

4.10. Zip and Email Report

Description: Running `./stealthguard.sh --sendemail` compresses all screenshots, process snapshots, and the audit log into a single ZIP archive and sends it as an email attachment to the configured recipient using `mutt`.

Code (stealthguard.sh):

```
1 ZIPFILE="/tmp/stealthguard_$(date +%Y-%m-%d_%H-%M-%S).zip"
2
3 zip -j "$ZIPFILE" "$VAULT"/capture_*.png \
4     "$VAULT"/processes_*.txt \
5     "$VAULT"/system_security_audit.log
6
7 echo "StealthGuard Security Report..." | \
8 mutt -s "StealthGuard Report $(date +%Y-%m-%d)" " " \
9     -a "$ZIPFILE" -- "$TO_EMAIL"
```

Listing 13: Zip and Email

Output:

```
(kali@kali)-[~/stealthguard]
$ ./stealthguard.sh --sendemail

=====
StealthGuard - IDS System
IIUC CSE | OS Lab Project
=====

[OK] /SecurityVault exists.
[MODE] Zip and send email.
[INFO] Zipping all captures ...
  adding: capture_2026-07-31_17-02-41.png (deflated 0%)
  adding: capture_2026-07-31_17-10-06.png (deflated 0%)
  adding: capture_2026-07-31_17-10-32.png (deflated 0%)
  adding: capture_2026-07-31_17-11-03.png (deflated 0%)
  adding: capture_2026-07-31_17-12-41.png (deflated 0%)
  adding: capture_2026-07-31_17-12-55.png (deflated 0%)
  adding: capture_2026-07-31_17-13-09.png (deflated 0%)
  adding: processes_2026-07-31_17-02-41.txt (deflated 53%)
  adding: processes_2026-07-31_17-10-06.txt (deflated 53%)
  adding: processes_2026-07-31_17-10-32.txt (deflated 52%)
  adding: processes_2026-07-31_17-11-03.txt (deflated 52%)
  adding: processes_2026-07-31_17-12-41.txt (deflated 52%)
  adding: processes_2026-07-31_17-12-55.txt (deflated 52%)
  adding: processes_2026-07-31_17-13-09.txt (deflated 52%)
  adding: system_security_audit.log (deflated 87%)
[OK] Zip created: /tmp/stealthguard_captures_2026-07-31_17-15-47.zip
[INFO] Sending zip email to c233456@ugrad.iiuc.ac.bd ...
```

Figure 8: Zip created and email sent confirmation

4.11. Screen Change Alert Email

Description: When the Python verifier outputs an ALERT for hash mismatch, the shell script automatically detects it and sends an immediate email alert to the configured recipient with the filename, new hash, and timestamp.

Code (stealthguard.sh):

```
1 PYTHON_OUTPUT=$(python3 "$VERIFY_SCRIPT" "$FILENAME" "$PROC_FILE")
2 echo "$PYTHON_OUTPUT"
3
4 if echo "$PYTHON_OUTPUT" | grep -q "ALERT"; then
5     HASH=$(echo "$PYTHON_OUTPUT" | grep "SHA-256" | awk '{print $2}')
6     send_alert_email "$FILENAME" "$HASH" "$TIMESTAMP"
7 fi
```

Listing 14: Hash Alert Email Trigger

Output:

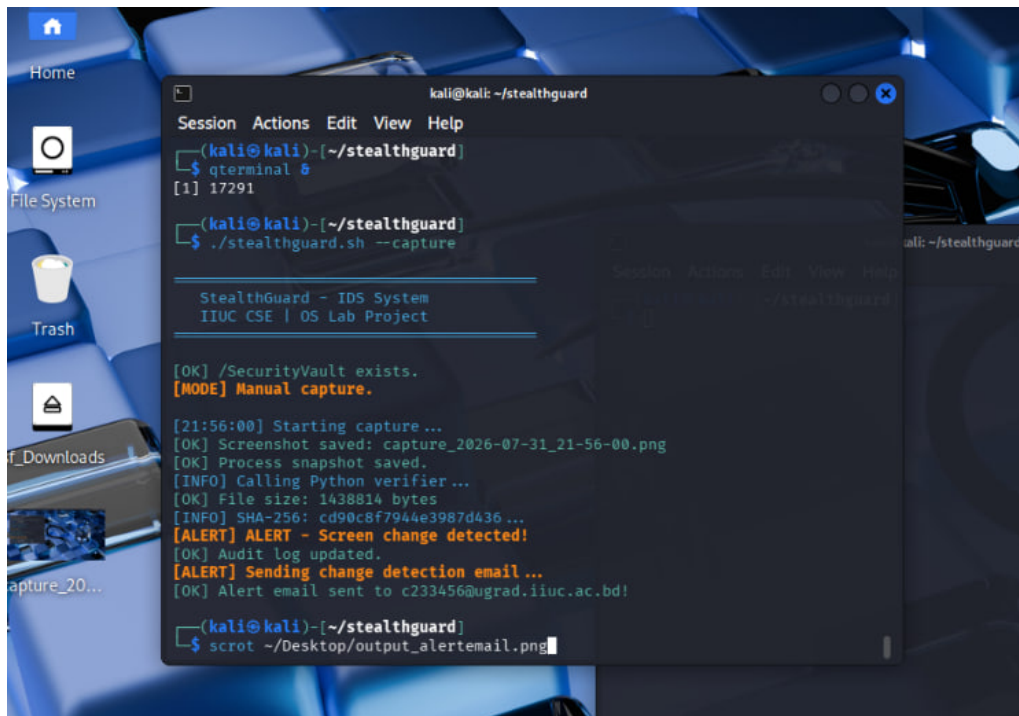


Figure 9: Alert email sent on screen change detection

The above output demonstrates the hash-based change detection working successfully. On the first capture, the system recorded the SHA-256 hash and saved it to `.previous_hash`. On the second capture, the newly calculated hash was different from the stored one, meaning the screen content had changed. The system automatically flagged this as an ALERT and triggered `send_alert_email()` without any user input, sending an email notification instantly to the configured recipient.

4.12. Real-Time Folder Watch and File Modification Alert

Description: Running `./stealthguard.sh --watchfolder` starts `inotifywait` which monitors the `/SecurityVault` directory in real time. The moment any file is modified, created, or deleted, the system instantly detects it, displays an alert in the terminal, and automatically sends an email notification containing the filename, event type, and timestamp.

Code (stealthguard.sh):

```

1 inotifywait -m "$VAULT" -e modify,create,delete \
2   --format '%e %f' |
3 while read -r event filename; do
4     if [ "$filename" == "system_security_audit.log" ]; then
5         continue
6     fi
7     echo -e "${RED}[ALERT] $event on: $filename${NC}"
8     send_modification_email "$filename" "$event"
9 done
10
11 send_modification_email() {
12     local filename="$1"
13     local event="$2"
14     echo "StealthGuard FILE MODIFICATION ALERT

```

```

15 WARNING: A file was modified inside /SecurityVault!
16 - File   : $filename
17 - Event  : $event
18 - Time   : $(date) " | \
19     mutt -s "ALERT - File Modified: $filename" -- "$TO_EMAIL"
20 }

```

Listing 15: Folder Watch and Modification Email

Output:

```

(kali㉿kali)-[~/stealthguard]
$ ./stealthguard.sh --watchfolder

=====
StealthGuard - IDS System
IIUC CSE | OS Lab Project
=====

[OK] /SecurityVault exists.
[MODE] Folder watch mode.
[INFO] Watching /SecurityVault for any file changes...
[INFO] Press Ctrl+C to stop.

Setting up watches.
Watches established.
[ALERT] CREATE detected on: capture_2026-06-24_15-27-35.png at 17:20:41
[ALERT] File modification detected: capture_2026-06-24_15-27-35.png
[OK] Modification alert sent to c233456@ugrad.iiuc.ac.bd!

```

Figure 10: Terminal 1 — --watchfolder running and waiting for changes

```

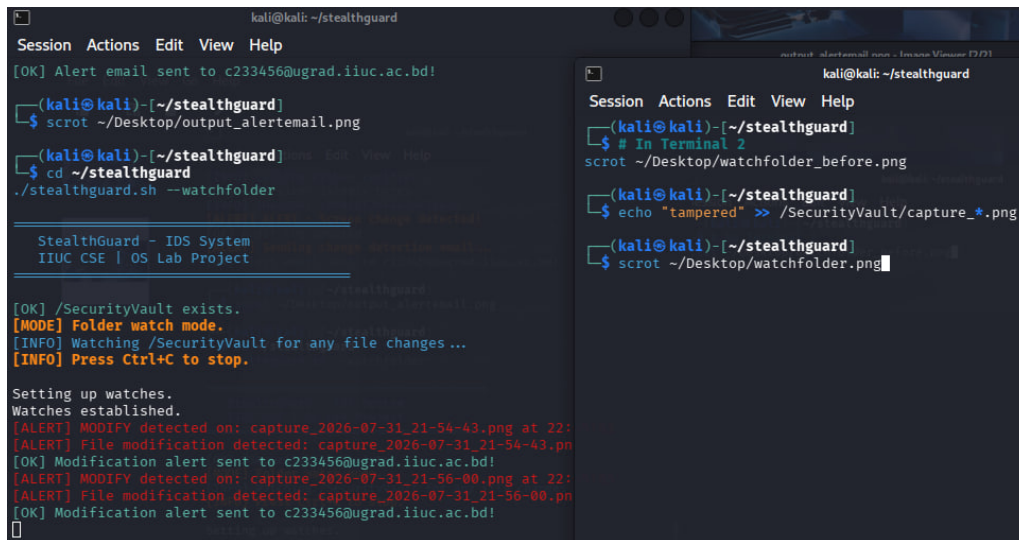
kali@kali: ~/Desktop
Session Actions Edit View Help
(kali㉿kali)-[~/Desktop]
$ echo "tampered" >> /SecurityVault/capture_2026-06-24_15-27-35.png

(kali㉿kali)-[~/Desktop]
$ echo "tampered" >> /SecurityVault/capture_*.png

(kali㉿kali)-[~/Desktop]
$

```

Figure 11: Terminal 2 — File tampered using echo "tampered" command



The image shows two terminal windows. The left window is titled 'kali@kali: ~/stealthguard' and shows the execution of the 'stealthguard.sh --watchfolder' command. It displays the program's header, mode selection, and the start of file monitoring. The right window, titled 'kali@kali: ~/stealthguard', shows a second terminal instance where the command 'echo "tampered" >> /SecurityVault/capture_*.png' is executed, followed by a screenshot of the 'watchfolder.png' file. This triggers an alert in the left window, which shows 'MODIFY detected' and 'File modification detected' messages, followed by an email alert being sent to 'c233456@ugrad.iuc.ac.bd!'.

Figure 12: File modification alert email received in inbox

As demonstrated above, the `--watchfolder` mode was started in Terminal 1, which activated `inotifywait` and began monitoring the directory. In Terminal 2, the command `echo "tampered"` was executed to simulate an unauthorized file modification. Terminal 1 immediately detected the `MODIFY` event and triggered an email alert to the recipient, as confirmed by the inbox screenshot. This demonstrates that any tampering with stored evidence files is detected and reported automatically without manual intervention.

5. Conclusion

StealthGuard successfully demonstrates a practical, dual-layer intrusion detection system built entirely on native Kali Linux tools without any external dependencies. By combining Bash shell scripting for system-level operations and Python 3 for cryptographic verification and persistent logging, the project implements a complete **Trigger** → **Capture** → **Verify** → **Record** security pipeline.

The system brings core Operating System concepts to life in a realistic and testable application. File system operations, process management, inter-process communication, cryptographic hashing, real-time event monitoring, and automated email notification are all demonstrated through working, functional code running directly on the operating system.

The addition of real-time folder monitoring using `inotifywait`, automated ZIP report delivery via `mutt`, SHA-256 hash-based change detection, and instant email alerts for both screen changes and file tampering elevates StealthGuard beyond a basic academic demonstration into a genuinely useful security tool. The project successfully covers all requirements of the Operating Systems laboratory including shell programming, process monitoring, file integrity verification, and inter-process communication between Shell and Python layers.

Ultimately, StealthGuard proves that meaningful system security monitoring can be implemented with minimal resources, using only the tools already present in a standard Kali Linux installation — making it both an effective academic exercise and a realistic prototype for real-world deployment.